

Missing stuff

Missed Content — From Lecture Notes

This file captures content from the class notes (pages 21–56 extract) that is **not already covered** (or not covered with equivalent detail) in the Layer 3, Layer 4, and Layer 6 notes. Everything here is examinable.

Layer 6 — ASN.1 Lecture Additions

BNF / EBNF / ABNF — Formal Syntax for Protocol Specification

The lecturer explicitly covered the formal grammar family that underpins protocol specification. While the existing Layer 6 notes mention that "ASN.1's structure is defined using a grammar", the lecture gave additional context about how this fits the broader Layer 6 picture.

Backus-Naur Form (BNF)

- Created by John Backus to describe the programming language Algol.
- Peter Naur also contributed, hence *Backus-Naur Form*.
- BNF became the standard mechanism for specifying formal languages.

EBNF — Extended BNF

- An extension of BNF; individual organisations/standards bodies extended it and made their own rules.
- Widely understood in Computer Science despite not being a single unified standard.

ABNF — Augmented BNF

- Another extension, more common in IETF RFCs.
- HTTP, for example, is specified using ABNF.

Lecture key point: Protocols are specified using some formal syntax — BNF, EBNF, or ABNF. ASN.1 itself sits at the far end of the complexity scale: it has a grammar to describe message structure *and* separate encoding rules to describe how that structure is physically serialised on the wire.

ASN.1 Encoding — The Lecturer's Notation: (T, L, Payload, 0)

The existing notes cover TLV (Type, Length, Value). The lecturer used a slightly different notation that is worth knowing exactly:

(T, L, Payload, 0)

- **T** — Type (describes what kind of data follows)
- **L** — Length (how long the payload is)
- **Payload** — the actual encoded value
- **0** — an indicator/terminator signalling the end of the payload

The lecturer noted this as a way of understanding the structure "at least 4 sets" of encoding rules:

Acronym	Full name	Brief description
BER	Basic Encoding Rules	The base standard
CER	Canonical Encoding Rules	Removes BER's optional alternatives
DER	Distinguished Encoding Rules	Further restricts CER; fixed lengths where possible

Lecture exact words: "Basic, canonical and distinguished" — the three most commonly used ASN.1 encoding rules. Know these three by name and order.

Layer 4 (6.5) — TCP, UDP, QUIC Lecture Additions

TCP — The Three Most Important Flags

The lecturer explicitly called out **three flags as the most important to know**:

```
ACK  }  
SYN  } 3 Important ones  
FIN  }
```

The others (URG, RST, PSH) were listed but not emphasised:

- **URG** — Urgent
- **RST** — Reset
- **PSH** — Push
- **FIN** — Terminate

Exam note: Know what ACK, SYN, and FIN are used for in the connection establishment and termination handshakes. The lecturer specifically stated: "Reliability — when does these 3 come in. TCP all about reliability."

TCP Connection Establishment — The "Length Question"

The lecturer highlighted the handshake flow as a *long question* for the semester test. The flow as noted on the whiteboard/slides:

SYN

SYN + ACK ← "lastcreate greeting" (now in established mode)

- At SYN: **separate timer** is started for the SYN. The lecturer noted SYN has its own timer — approximately 60–70 seconds — "send out SYN – hear nothing, time out, got nothing."
- The server can turn into a client (active/passive open distinction from the state graph).

Lecture fact — SYN has its own timer: This is the **connection-establishment timer** (sometimes called the SYN timer). It is separate from the five named TCP timers (retransmission, acknowledgement, persistence, keepalive, quiet). It fires after approximately 60–70 seconds if the SYN receives no response, and the connection attempt is abandoned. This is NOT in the standard five-timer list but was called out in lectures.

TCP — TIME-WAIT State (Lecturer's Explanation)

The lecturer described TIME-WAIT explicitly as:

"Terminated, closed, waiting for all messages to finish before moves on. Port remains unavailable for a period of time. Return + Time trip to be."

Key lecturer points:

- **Reuse port immediately?** — No, because a packet from the previous connection could still be in transit ("apart of previous connection").
- Waits "more or less **twice as long** as for anything out of use to arrive."
- "Then go to close."
- The node sends FIN and is expecting FIN+ACK.

The lecturer used the state-graph diagram (from the textbook) and the note: "*navigate path*" — meaning you are expected to be able to trace the path through the TCP state graph, not just name the states.

TCP — Window Advertisement (Lecturer's Worked Example on Whiteboard)

The lecturer worked through a window advertisement scenario on the whiteboard. The exact values shown on the projected slide were:

ACK = 75

SEQ = 75 LEN = 5 → (A to B)

ACK = 150

```
SEQ = 80    LEN = 10
ACK = 150
SEQ = 80    LEN = 20
ACK = 150
SEQ = -
```

The accompanying notes explain the concept:

- **"Running out of buffer space"** → tells the other party what window size is.
- The **Window Advertisement** tells the other side "how much capacity I have left."
- **Buffer full** → browser sending pages, buffer gets full → window advertisement drops to 0.

Lecture summary: The window advertisement is a field in the TCP header. Its purpose is to advertise how much free buffer space remains at the receiver. When the receiver's buffer fills up (e.g. a web browser sending many pages), the window advertisement drops. The sender must stop sending when it would exceed the advertised window. This is the mechanism that implements TCP flow control.

TCP — netstat -n Flag

The lecturer demonstrated `netstat` live and highlighted one specific flag:

`netstat -n` — gives **numeric addresses** (IP addresses in dotted-decimal, not domain names).

Without `-n`, `netstat` may try to do reverse-DNS lookups on each address (slow). With `-n`, it displays raw IP addresses and port numbers immediately.

Other flags demonstrated but not specifically MCQ-targeted: `--help`, options to show UDP connections.

The lecturer showed the Windows version (`netstat` in a Command Prompt window), which displays:

Active Connections

Proto	Local Address	Foreign Address	State
TCP	[address]:[port]	[address]:[port]	ESTABLISHED
TCP	[address]:[port]	[address]:[port]	TIME_WAIT

Exam interpretation note: When you see `ESTABLISHED` in `netstat` output, "what does it mean if same here twice?" (the lecturer's phrasing) — it means both sides of the connection are the same machine (loopback), but the ports differ. This is normal for inter-process communication on one machine.

Layer 4 — QUIC Lecture Additions

QUIC — Speed: Two Specific Mechanisms Named in Lecture

The lecturer listed exactly **two things QUIC does to improve speed** (beyond the general "reduces latency" answer):

1. **Piggybacking — "going together"** ("If piggyback so go together") — combining multiple frames into one packet, compressing handshakes.
2. **Interleaving streams** — "loading website, loading image ... interleave certain things, image, the HTML whilst image ... did dogs — wait for whole image." This is the head-of-line blocking elimination.

The lecturer also mentioned **supporting migration of a client to a new network** as a QUIC speed/continuity feature (listed with "compressing handshakes" and "interleaving streams of HTML page").

Exact lecture list — QUIC improves speed by:

1. Compressing/combining handshakes
2. Interleaving streams of HTML and other resources
3. Supporting migration of a client to a new network

QUIC — TLS 1.2 vs TLS 1.3 Context

The lecturer's notes explicitly label the old protocol as **TLS 1.2/3** with the annotation "2 RTT before communicate → a lot of time wasted." The QUIC diagram label says **TLS 1.2 3** crossed through, with "1 RTT" on the QUIC side.

Key comparison the lecturer wanted students to understand:

- **TCP + TLS (old):** 2 RTT before any encrypted application data can flow. "A lot of time wasted."
- **QUIC:** 1-RTT for a new connection; **0-RTT for a reconnection** (using cached keys).

QUIC — Server Can Communicate at Half-RTT ($\frac{1}{2}$ RTT)

The lecturer's notes on the QUIC handshake diagram (page 10 of the PDF) contain:

"Node on right-hand side wants to communicate at $\frac{1}{2}$ RTT." "1 RTT = worst case scenario." "Communicating at 0-RTT. Communicate in encrypted fashion. Introduce communication. → at half RTT → use new setup. Old key for half RTT."

This describes a detail not fully captured in the existing notes:

The server can send 1-RTT application data at t_1 — which is $\frac{1}{2}$ RTT from the perspective of the full exchange. While the client's 1-RTT data is only available at t_2

(after one full round trip), the server already has enough cryptographic material at t_1 (from the client's initial packet at t_0) to begin sending encrypted 1-RTT data immediately. So from t_0 to t_1 is half a round trip — the server is already communicating encrypted.

Party	Soonest can send encrypted application data	Why
Client (0-RTT)	t_0	Cached keys from previous session
Server (1-RTT)	t_1 ($\frac{1}{2}$ RTT from client's perspective)	Server has client's crypto material from initial packet
Client (1-RTT)	t_2	Must receive server's reply first

The lecturer noted: "Why only use that key for half RTT? Use as little as possible." — meaning the old/cached key is used only for the brief 0-RTT window; after the handshake, a fresh session key takes over.

QUIC — Packet Type Table (QUICv1 vs QUICv2)

The lecturer specifically circled this table on the projected slide and wrote: **"understand this."**

Packet Type	QUICv1 (2-bit code)	QUICv2 (2-bit code)
Initial	00	01
0-RTT	01	10
Handshake	10	11
Retry	11	00

Why this matters for ossification: QUICv1 uses 00 for Initial packets; QUICv2 uses 01. A firewall that looks for 00 to identify QUIC Initial packets will incorrectly classify QUICv2 Initial packets (coded as 01) and miss them entirely. This is a deliberate design decision — it makes it impossible for intermediaries to reliably detect and hardcode responses to QUIC packet types, preventing ossification.

Exam implication: QUIC version 3 will use "entirely different versions" — the lecturer wrote: "Version 3 use entirely diff version." Future versions will continue this pattern of unpredictability.

QUIC — Short Header (1-RTT Packets)

The existing notes cover the **long header** in detail. The lecturer also showed the **short header** used for 1-RTT (application data) packets. The short header is much simpler:

Header Form	Fixed Bit	Key Phase (1)	...
	Destination Connection ID (0..160)		
	Packet Number (8..32)		
	Payload		

Key points about the short header:

- **Header Form bit = 0** (as opposed to 1 for long headers)
- **Fixed Bit** is always 1
- **Key Phase bit** — indicates which of two encryption keys is currently in use (allows keys to be rotated during the connection)
- **No Source Connection ID** — once the connection is established, only the destination ID is needed
- **No Version field** — only long headers carry the version

Exam implication: Long headers are used for Initial, Handshake, 0-RTT, and Retry packets. Short headers are used for **1-RTT (application data)** packets. The presence or absence of the Header Form bit = 1 (long) vs 0 (short) tells you which you are looking at.

QUIC — Wireshark Capture Details (From Lecture Slide)

The lecturer showed a live Wireshark capture of a QUIC Initial packet. The visible fields in the capture were:

User Datagram Protocol, Src Port: 9296, Dst Port: 443

QUIC IETF

= Header Form: Long Header (1)

= Fixed Bit: True

...00 = Packet Type: Initial (0)

...2.. = Packet Number Length: 1 byte (01)

Version: 0x00000001

Destination Connection ID Length: 8

Destination Connection ID: a85d1904c3f73c7b

Source Connection ID Length: 0

Token Length: 0

Token: <MISSING>

Length: 10

Token: <UNKNOWN>

Packet Number: 1851

Payload:

CRYPTO

PADDING

PING

PADDING

CRYPTO
PING

What this shows: Initial packets are sent to a **well-known UDP port (443 for HTTPS)**. The Source Connection ID Length is 0 (empty) at first — the server has not yet chosen an ID. The payload contains CRYPTO frames (TLS handshake data) and PADDING.

The lecturer also showed a **server's first response packet** containing:

```
QUIC IETF
Version: 0x00000001
Source Connection ID: (length 5, some value)
...
Payload: STREAM, ACK, PADDING, CRYPTO, PING
```

Layer 3 — Routing / Layer 3 Lecture Additions

(The lecture notes provided cover primarily Layer 4 and Layer 6. The Layer 3 pages from the lecture notes (pages 14–36 of the extract) consist mostly of the special IPv4 addresses table photo, which is already fully captured in the Layer 3 notes.)

Special IPv4 Addresses — Lecturer Confirmed Scope

The lecturer projected Table 7.1 directly. This confirms the following table is examinable exactly as shown (already in Layer 3 notes, confirmed here for completeness):

Address	Meaning
0.0.0.0/0	Default gateway entry
0.0.0.0/32	Placeholder for 'this' computer
10.0.0.0/8	Private address block
100.64.0.0/10	The shared address space (for CGN)
169.254.0.0/16	Link-local addresses
172.16.0.0/12	Private address block
192.168.0.0/16	Private address block
255.255.255.255/32	'Local' broadcast

Summary of New Items Not Previously in Any Note File

#	Topic	Layer	Status in existing notes
1	BNF / EBNF / ABNF explained with their history	L6	Not included (only "grammar" mentioned generically)
2	ASN.1 encoding notation: (T, L, Payload, 0)	L6	Covered as TLV but not this exact notation
3	Three encodings by name in order: BER, CER, DER	L6	Covered, but lecturer used simpler mnemonic
4	SYN has its own connection-establishment timer (~60–70s)	L4	Not mentioned at all
5	ACK, SYN, FIN are the three most important TCP flags	L4	All named but not emphasised as a group of three
6	TIME-WAIT waits approximately 2× round-trip time	L4	Stated as "twice the maximum segment lifetime" but not RTT framing
7	<code>netstat -n</code> gives numeric addresses	L4	Not mentioned
8	QUIC's exact two speed mechanisms as listed by lecturer	L4	Covered separately but not as a named pair
9	Server can send 1-RTT data at t_1 ($\frac{1}{2}$ RTT from client's perspective)	L4	Missing — notes only say "client" for 1-RTT at t_2
10	QUIC packet type codes table (QUICv1 vs QUICv2 with exact bit patterns)	L4	Version numbers mentioned but exact table not included
11	Short QUIC header structure (Header Form=0, Key Phase bit, no Source ID)	L4	Only long header covered
12	Key Phase bit in short QUIC header (key rotation indicator)	L4	Not mentioned
13	Wireshark capture details of QUIC Initial packet	L4	Only mentioned conceptually
14	QUIC version 3 will use entirely different version number pattern	L4	Not mentioned